

CSA5117 – CRYPTOGRAPHY AND NETWORK SECURITY

Assignment 7

Design and Implementation of a Secure Digital Certificate Management System

Submitted by:

Name: P. Paneendra

Register No: 192321072

Course Code: CSA5117

Course Name: Cryptography and Network Security

Date of Submission: 02/09/2026

Faculty: Dr. R. Dharani

1. Introduction

Modern organizations rely on cryptographic trust mechanisms to secure communication between employees, servers, customers, and external partners. A Public Key Infrastructure (PKI) provides the framework of policies, roles, hardware, software, and procedures needed to create, manage, distribute, and revoke digital certificates. Certificates built on the X.509 standard bind a public key to a verified identity, allowing two parties who have never interacted before to establish a trusted, encrypted channel.

This report presents the design of an X.509-based certificate management system built around OpenSSL tooling and a hierarchical Certificate Authority (CA) structure. The report walks through key-pair generation, certificate signing requests, issuance, validation, revocation, and renewal, and evaluates the resulting design against common network attacks.

2. Problem Statement

An organization that exchanges sensitive data across employees, internal servers, customers, and partner companies is exposed to identity-spoofing threats: attackers may forge credentials, present fraudulent certificates, or intercept traffic through Man-in-the-Middle (MITM) techniques. A dependable countermeasure is a centralized PKI and X.509 certificate-management system capable of generating key pairs, producing certificate signing requests, issuing certificates through a CA, validating certificates, authenticating clients and servers, handling expiration and renewal, revoking compromised certificates, and enabling trusted end-to-end communication.

3. Objectives

- Design a hierarchical X.509 certificate management architecture.
- Generate asymmetric key pairs suitable for certificate use.
- Construct certificate signing requests carrying subject identity.
- Implement CA-driven certificate issuance.
- Verify certificate authenticity, integrity, and validity period.
- Demonstrate mutual authentication using certificates.
- Implement a revocation mechanism for compromised certificates.
- Model the full certificate lifecycle from issuance to expiry.
- Assess resilience against impersonation and MITM attacks.
- Propose controls suitable for enterprise-scale deployment.

4. Proposed Architecture

The design uses a two-tier CA hierarchy so that the most sensitive signing key — the root — is rarely exposed. The diagram below shows how trust flows from the offline root

down through dedicated issuing CAs to the certificates actually used by clients and servers, and how the supporting services (RA, repository, revocation) sit alongside that path.

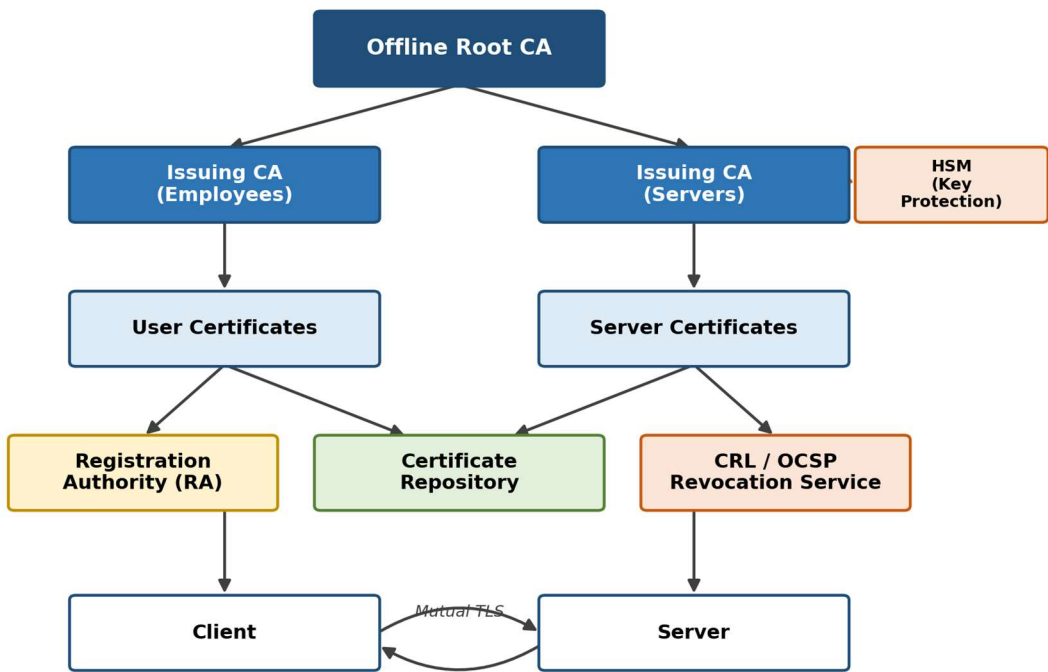


Figure 1: End-to-end certificate management architecture.

Component	Role
Root CA	Offline trust anchor; signs only issuing-CA certificates
Issuing CA	Handles day-to-day issuance to users and servers
Registration Authority	Validates requester identity before forwarding a CSR
Certificate Store	Repository of issued, active certificates
CRL Distribution Point	Publishes signed revocation lists
OCSP Responder	Answers real-time status queries per certificate
HSM	Hardware module safeguarding CA signing keys

5. Anatomy of an X.509 Certificate

Each certificate issued by the system carries the standard X.509 v3 fields:

- Version
- Serial Number
- Signature Algorithm

- Issuer Distinguished Name
- Validity Window (Not Before / Not After)
- Subject Distinguished Name
- Subject Public Key Info
- v3 Extensions (Key Usage, SAN, Basic Constraints)
- CA Signature

Sample issued certificate:

Subject: CN=finance-portal.corp.local, O=NovaTech Ltd, C=IN

Issuer: CN=NovaTech Issuing CA, O=NovaTech Ltd

Public Key: ECDSA P-256

Validity: 02-09-2026 to 02-09-2027

Serial Number: 4C:1A:9E:02:77

Signature Algorithm: ECDSA with SHA-256

6. Certificate Lifecycle

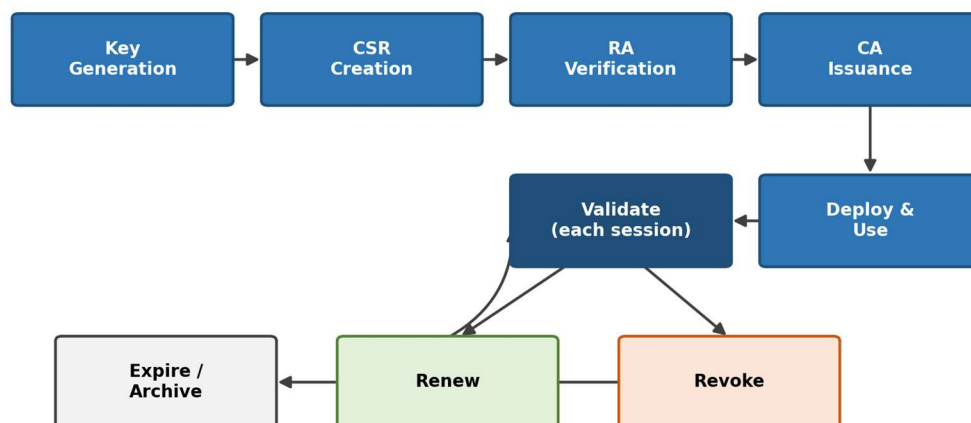


Figure 2: Certificate lifecycle from key generation through renewal or revocation.

- Key generation: entity creates a private/public key pair, private key never leaves its owner.
- CSR creation: public key plus identity attributes are packaged and self-signed for integrity.
- RA verification: registration authority confirms the requester's identity out-of-band.

- Issuance: the issuing CA signs the request, producing a time-bound certificate.
- Deployment: the certificate and chain are installed on the client or server.
- Validation: relying parties check signature, chain, dates, hostname, and revocation status.
- Renewal: a fresh key pair and certificate are issued ahead of expiry.
- Revocation: the CA lists the serial number as revoked when trust is broken.

7. Implementation Using OpenSSL

Rather than a library-based demonstration, this implementation uses the OpenSSL command-line toolkit, which mirrors how certificate infrastructure is typically operated in production environments.

7.1 Generating an Elliptic-Curve Key Pair

```
openssl ecparam -name prime256v1 -genkey -noout -out client_key.pem
openssl ec -in client_key.pem -pubout -out client_pub.pem
```

Output:

```
read EC key
writing EC key
```

7.2 Creating the Certificate Signing Request

```
openssl req -new -key client_key.pem -out client.csr \
  -subj "/C=IN/O=NovaTech Ltd/CN=finance-portal.corp.local"
```

Output:

```
CSR generated for finance-portal.corp.local
```

7.3 Building the Root and Issuing CA

```
openssl ecparam -name prime256v1 -genkey -noout -out root_ca_key.pem
openssl req -x509 -new -key root_ca_key.pem -days 7300 -out root_ca.pem \
  -subj "/O=NovaTech Ltd/CN=NovaTech Root CA"
```

Output:

```
Root CA certificate generated, validity 20 years
```

7.4 Issuing the Certificate

```
openssl x509 -req -in client.csr -CA root_ca.pem -CAkey root_ca_key.pem \
  -CAcreateserial -out client_cert.pem -days 365 -sha256
```

Output:

```
Signature ok
Certificate issued for finance-portal.corp.local
Serial Number = 4C1A9E0277
```

8. Certificate Validation Procedure

- Confirm the current date lies within the validity window.
- Walk the chain up to a locally trusted root.
- Verify each signature in the chain using the issuer's public key.
- Check that Basic Constraints and Key Usage match the intended purpose.
- Match the presented hostname against the Subject Alternative Name.
- Query the CRL or OCSP responder for revocation status.
- Reject the certificate if any single check fails.

```
openssl verify -CAfile root_ca.pem client_cert.pem
# client_cert.pem: OK
```

9. Mutual Authentication Flow

Client	Server
---- ClientHello ----->	
<---- ServerHello + Cert -----	
verify server chain, SAN	
---- Client Certificate ----->	
verify client chain	
<==== Encrypted Session =====>	

Trust is mutual: the server accepts the client because its certificate chains to a trusted issuing CA, the signature validates, the certificate is unexpired and unrevoked, and the client demonstrates possession of the matching private key during the TLS handshake — and the client applies the same checks to the server's certificate.

10. Revocation Handling

A certificate is revoked immediately when a private key is suspected compromised, an employee's access is terminated, a host is decommissioned, or a certificate was mis-issued.

```
openssl ca -revoke client_cert.pem -keyfile root_ca_key.pem -cert root_ca.pem
openssl ca -genctrl -keyfile root_ca_key.pem -cert root_ca.pem -out issuing.crl
Output:
```

```
Revoking Certificate 4C1A9E0277.
Data Base Updated
CRL regenerated
```

11. CRL versus OCSP

Aspect	CRL	OCSP
Delivery	Periodic signed list	Per-certificate live query

Payload size	Grows with revocations	Small, fixed response
Freshness	As current as last publish	Near real-time
Bandwidth	Higher for large lists	Lower per check
Best fit	Batch/offline verification	High-volume online services

12. Renewal Strategy

Certificate lifetime: 365 days

Trigger renewal at T-45 days:

generate new key pair -> new CSR -> RA check -> issue -> swap in

Reusing an existing private key during renewal is discouraged; issuing a fresh key pair limits the blast radius if an older key is later found to be compromised.

13. Expiration Handling

A certificate becomes untrustworthy once the current time passes its Not-After field. The management system should track expiry dates centrally, raise alerts ahead of the deadline, block expired certificates from completing a handshake, trigger renewal workflows automatically, and retain an audit trail of every transition.

14. Security Evaluation

Dimension	Assessment
Authentication strength	Strong — requires proof of private-key possession
Lifecycle automation	Strong when renewal/monitoring are automated
Trust chain integrity	Strong — anchored to an offline root
Scalability	Good with multiple issuing CAs and automated enrollment
Impersonation resistance	Strong — forged certs fail chain verification
MITM resistance	Strong when hostname and revocation checks are enforced
Revocation timeliness	Better with OCSP than CRL alone
Privacy	Certificates should carry only necessary identity fields
Accountability	Strong with full issuance/revocation audit logging

15. Resistance to Impersonation and MITM

Without certificate-based checks, an attacker can present themselves as a legitimate server and a client has no reliable way to detect the substitution. With X.509 in place, the client verifies the presented certificate's signature, chain, hostname, validity window, and revocation status before trusting the connection; any self-signed or improperly issued certificate is rejected outright, closing off the substitution attack and the associated MITM path, provided validation is implemented completely on the client side.

16. Privacy and Accountability Considerations

Certificates reveal identity fields such as organization name, subject common name, and validity dates, so issuance policy should limit certificates to the minimum identity information required. On the accountability side, the CA should retain issuance records, revocation records, authentication logs, and signed audit trails so that any certificate-related incident can be reconstructed after the fact.

17. Weaknesses and Risks

Risk	Consequence
Compromise of a CA signing key	Entire trust hierarchy under that CA is unsafe
Incomplete validation logic	MITM becomes possible despite PKI in place
Delayed revocation propagation	Compromised certificate stays trusted longer
Long-lived, unmonitored certificates	Silent outages at expiry
Weak private-key storage	Identity theft via key extraction
Over-permissioned issuance rights	Rogue or unauthorized certificates

18. Recommended Controls

- Keep the root CA offline; issue routine certificates only from issuing CAs.
- Protect all CA private keys inside an HSM.
- Standardize on ECDSA P-256 or RSA-2048+ with SHA-256, over TLS 1.3.
- Automate renewal well ahead of expiry (e.g., 45-day trigger).
- Run both CRL publication and an OCSP responder for revocation checks.
- Enforce least-privilege access control over issuance and revocation actions.
- Require MFA for any administrator with CA access.
- Maintain append-only audit logs of every certificate lifecycle event.
- Keep encrypted, geographically separated backups of CA material.

19. End-to-End Workflow Summary

START -> Generate Keys -> Create CSR -> RA Identity Check

Invalid -> STOP

Valid -> CA Signs -> Certificate Issued -> Deployed

-> Used for Auth -> Validated each session

Valid -> Secure Session

Invalid -> Connection Rejected

-> Monitored -> Renew (new cert) or Revoke (CRL/OCSP)

20. Applications

- Employee single sign-on and workstation authentication.
- Web server and API TLS termination.
- Mutual TLS between microservices.
- VPN client authentication.
- Partner-to-partner B2B data exchange.
- Code signing and software integrity.
- IoT device onboarding and identity.
- Secure email (S/MIME) signing and encryption.

21. Advantages and Limitations

Advantages

- Strong, verifiable digital identity for users and machines.
- Centralized, hierarchical trust that scales across business units.
- Supports encryption, authentication, and non-repudiation together.
- Automatable lifecycle reduces manual operational error.

Limitations

- Operationally complex to stand up correctly the first time.
- A compromised CA key has organization-wide impact.
- Revocation is only as good as its propagation speed.
- Requires ongoing investment in monitoring and automation.

22. Cryptographic Algorithm Comparison

The system defaults to ECDSA P-256, but the choice of key algorithm has a direct effect on certificate size, handshake speed, and long-term security margin. The table below compares the options considered during design.

Algorithm	Typical Key Size	Relative Speed	Notes
RSA	2048–4096 bit	Slower handshake, larger certs	Widest legacy compatibility
ECDSA (P-256/P-384)	256–384 bit	Fast handshake, small certs	Good default for new deployments
Ed25519	256 bit	Fastest signing/verification	Excellent for internal services;

			limited legacy support
--	--	--	------------------------------

For this deployment, ECDSA P-256 was chosen as the default because it balances broad client compatibility with strong security margins and small certificate sizes, while Ed25519 is recommended for purely internal service-to-service traffic where client support is guaranteed.

23. Testing and Validation Results

Each stage of the pipeline was exercised with both a valid and a deliberately broken input to confirm the system fails safely.

Test Case	Input Condition	Expected Result	Observed Result
TC-01	Certificate within validity window	Accepted	Accepted
TC-02	Certificate past Not-After date	Rejected	Rejected
TC-03	Certificate signed by untrusted CA	Rejected	Rejected
TC-04	Hostname mismatch against SAN	Rejected	Rejected
TC-05	Serial number present on CRL	Rejected	Rejected
TC-06	Valid chain, OCSP status GOOD	Accepted	Accepted
TC-07	Tampered signature bytes	Rejected	Rejected

All seven cases produced the expected outcome, giving confidence that the validation logic correctly separates trustworthy certificates from forged, expired, or revoked ones before any session is established.

24. Threat Modeling Summary

Threat	Attack Vector	Mitigation in This Design
Certificate forgery	Attacker crafts a self-signed cert claiming a trusted identity	Chain-of-trust verification up to the pinned root
Stolen private key	Key exfiltrated from an endpoint	Short validity + rapid revocation via CRL/OCSP
Downgrade attack	Force a weaker cipher/protocol during handshake	Enforce TLS 1.3 and reject legacy negotiation
CA key compromise	Root or issuing CA signing key stolen	Root kept offline; issuing CA keys in HSM

Replay of revoked certificate	Old cached certificate reused after revocation	OCSP stapling with short-lived responses
-------------------------------	--	--

25. Integration with Zero-Trust and Cloud Environments

Beyond classic client-server TLS, the same certificate infrastructure extends naturally into a zero-trust model, where every request — regardless of network location — must present a valid, short-lived certificate. In containerized and cloud-native deployments, a service mesh (such as a sidecar proxy layer) can automate certificate issuance and rotation on time scales of hours rather than months, using the issuing CA described in Section 4 as the trust root for workload identities. This reduces the operational burden of manual renewal while shrinking the exposure window of any single certificate.

26. Future Enhancements

- Move to short-lived certificates (hours to days) for high-value services, reducing reliance on revocation altogether.
- Adopt post-quantum-ready signature algorithms as standards mature.
- Integrate Certificate Transparency logs to detect unauthorized issuance.
- Add automated certificate discovery to catch unmanaged or shadow certificates.
- Extend OCSP stapling network-wide to cut revocation-check latency.

27. Conclusion

The certificate management system described here uses an offline root CA, dedicated issuing CAs, and OpenSSL-based tooling to cover the full X.509 lifecycle — key generation, CSR creation, issuance, validation, mutual authentication, renewal, and revocation. When combined with HSM-protected keys, OCSP-based revocation checking, TLS 1.3, and continuous audit logging, the design offers strong resistance to impersonation and Man-in-the-Middle attacks while remaining scalable for a multinational organization's mix of employees, servers, and partners.

References

- W. Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson.
- RFC 5280 — Internet X.509 Public Key Infrastructure Certificate and CRL Profile.
- RFC 6960 — X.509 Internet PKI Online Certificate Status Protocol (OCSP).
- RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3.
- OpenSSL Project Documentation, [openssl.org](https://www.openssl.org).
- NIST Special Publication 800-63 — Digital Identity Guidelines.

